

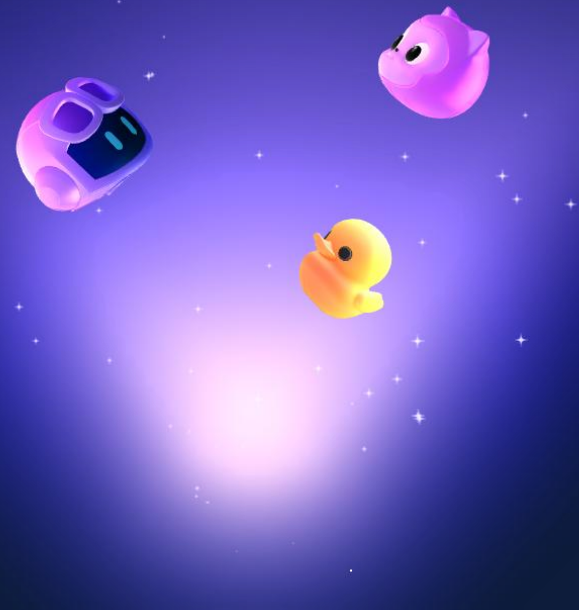


GitHub

Trabajo Remoto y Colaborativo

Informática II — UTN FRH

Profesor: Weber Federico



Git vs GitHub

2

No son lo mismo — es la confusión más común

Git

- Sistema de control de versiones
- Funciona en tu computadora (LOCAL)
- Guarda historial de cambios
- Lo que vimos la clase pasada

GitHub

- Plataforma/Servicio que aloja proyectos Git
- Funciona en la NUBE (remoto)
- Permite colaborar con otros
- Lo que vemos hoy

¿Por qué GitHub?

3

Backup en la nube

Tu código no se pierde si se rompe tu PC

Colaboración

Trabajar en equipo en el mismo proyecto

Portfolio profesional

GitHub es tu CV de programador

Entrega de TPs

Los trabajos prácticos se entregan vía GitHub

"GitHub es como una red social para desarrolladores"



Configuración de tu cuenta de GitHub

Correo universitario + GitHub Education

Cómo configurar tu cuenta

Usá tu mail personal como principal y el de la universidad como secundario



Mail principal

Usen su mail personal (Gmail, Outlook, etc.) como correo principal de GitHub.



Mail secundario

Agreguen mail de la universidad (.edu / institucional) como correo secundario.



¿Por qué hacerlo así?

Si usan el mail de la universidad como principal y algún día pierden acceso (se gradúan, cambian de carrera, etc.), pueden quedar sin acceso a su cuenta de GitHub.

Con el personal como principal, siempre van a poder recuperar la cuenta.

GitHub Student Developer Pack

Beneficios gratuitos por tener un mail universitario verificado



GitHub Copilot Student

Asistente de código con IA. Autocompletado inteligente directo en tu editor.



GitHub Codespaces

Entorno de desarrollo en la nube. 180 horas/mes gratis. Programá desde cualquier lugar.



Certificación gratis

1 voucher para rendir GitHub Foundations o GitHub Copilot Certification (verificar fecha de vencimiento).



+100 herramientas

Azure \$100, dominios gratis, Notion, 1Password, cursos en FrontendMasters y más.

No vamos a usar estas herramientas en la cursada, pero están disponibles para que las exploren por su cuenta.

¿Cómo activar los beneficios?

1

Crear cuenta en github.com con tu mail personal

2

Ir a Settings → Emails → Agregar tu mail universitario y verificarlo

3

Ir a education.github.com/discount_requests/application

4

Completar el formulario con tu mail universitario y foto de credencial/constancia

La aprobación puede tardar unos días. Una vez aprobado, activar Copilot desde Settings → Copilot.

education.github.com/pack

II PAUSA — Creemos las cuentas ahora

Explorando GitHub

9

Pestañas de tu perfil:

Overview

Tu página principal

Repositories

Tus proyectos

Projects

Tableros de tareas

Packages

Paquetes publicados

Stars

Repos favoritos

Explore: descubran proyectos

- github.com/explore — proyectos populares y trending
- Topics — buscar por tema (Python, Arduino, machine-learning...)
- Stars — guarden repos que les interesen como favoritos

Busquen el repositorio de la cátedra y agréguenlo a favoritos ★

Tu primer repositorio: el perfil

GitHub tiene un repo especial:

Si tu usuario es **estudiantex**, creá un repo llamado **estudiantex**

- ▶ Debe ser público
- ▶ El README.md aparece como tu página de perfil
- ▶ Es como tu CV

Ejemplo de README.md:

```
# Hola, soy Sebastián 🙋
```

```
Estudiante de Ingeniería Electrónica - UTN FRH
```

```
## Tecnologías
```

- C/C++
- Git & GitHub

Dos formas de crear un repo:

- ① Crear en GitHub y clonar
(recomendado para empezar)
- ② Crear local con git y subir
(lo veremos en breve)

Markdown: el lenguaje de texto enriquecido que usa GitHub

11

Todo texto en GitHub se escribe en Markdown

Escribís esto:

```
# Título principal
## Subtítulo

Texto normal, **negrita**, *cursiva*

- Lista con viñetas
- Otro item

```
código en bloque
```

[link](https://...)
```

Se ve así:

Título principal

Subtítulo

Texto normal, **negrita**, *cursiva*

- Lista con viñetas
- Otro item

```
código en bloque
```

[link](#)

Todo repositorio debería tener un README.md

- GitHub lo renderiza automáticamente en la página del repo
- Describe qué hace el proyecto, cómo instalarlo, cómo usarlo
- Es lo primero que ve alguien que visita tu repo

Documentación oficial:

docs.github.com/es

Completa, en español, con guías paso a paso

Autenticación: HTTPS vs SSH

¿Cómo nos conectamos a GitHub desde la terminal de trabajo, donde usamos GIT?

HTTPS

- Pide usuario y token (PAT)
- Cada vez que hacés push o pull
- Más simple al inicio
- Molesto a la larga

SSH ✓ recomendado

- Configurás UNA vez por PC
- Después funciona automáticamente
- Más seguro (criptografía)
- El estándar de la industria

SSH usa un **par de claves**: una pública (la subís a GitHub) y una privada (queda en tu PC)

No necesitan entender la criptografía detrás, solo seguir los pasos

Paso 1: Generar clave SSH

Abrimos la terminal y ejecutamos:

```
ssh-keygen -t ed25519 -C "tu@email.com"
```

¿Qué hace cada parte?

ssh-keygen — genera un par de claves

-t ed25519 — tipo de algoritmo (moderno y seguro)

-C "tu@email.com" — comentario para identificar la clave

Se crean dos archivos:

~/.ssh/id_ed25519 — clave PRIVADA (NUNCA compartir)

~/.ssh/id_ed25519.pub — clave pública (se sube a GitHub)

Paso 2 (opcional): Configurar SSH

Si usaste el nombre por defecto (**id_ed25519**), este paso NO es obligatorio. SSH lo encuentra automáticamente. Pero es buena práctica configurarlo.

Creamos (o editamos) el archivo `~/.ssh/config`:

```
# GitHub
Host github.com
  HostName github.com
  User git
  IdentityFile ~/.ssh/id_ed25519
```

¿Para qué sirve este archivo?

- ▶ Útil si tenés VARIAS claves SSH (trabajo, personal, etc.)
- ▶ Le dice a SSH qué clave usar para cada servidor
- ▶ Si solo tienen una clave con el nombre por defecto, pueden saltarlo

Paso 3: Agregar clave a GitHub

En GitHub:

1. Settings → SSH and GPG keys → New SSH key
2. Pegá el contenido de tu clave pública:

```
$ cat ~/.ssh/id_ed25519.pub  
ssh-ed25519 AAAA...largo... tu@email.com
```

Verificamos que funcione:

```
$ ssh -T git@github.com  
Hi alumnox! You've successfully authenticated, but GitHub  
does not provide shell access.
```

Si ves ese mensaje con tu usuario, ¡ya estás conectado!

Cambiar remote de HTTPS a SSH (opcional)

17

Si ya tenés un repo clonado con HTTPS, podés cambiar a SSH:

Verificá tu remote actual:

```
$ git remote -v
origin  https://github.com/user/repo.git (fetch)
origin  https://github.com/user/repo.git (push)
```

Cambialo a SSH:

```
$ git remote set-url origin git@github.com:user/repo.git
$ git remote -v
origin  git@github.com:user/repo.git (fetch)
```

Solo necesitás hacer esto si tu remote actual es HTTPS

A. Subir un proyecto local a GitHub

Solo cuando empezás un proyecto con git init (UNA VEZ POR REPO)

```
git remote add origin git@github.com:<tu-usuario>/<tu-repo>.git
```

→ Conecta tu repo local con el remoto. Antes, crear el repo vacío en GitHub (sin README).

```
git branch -M main
```

→ Renombra tu rama actual a "main" (el nombre por defecto en GitHub).

```
git push -u origin main
```

→ Sube la rama main al remoto. -u guarda la asociación para el futuro.

Si clonaste un repo (git clone), NO necesitás nada de esto. Clone ya configura todo.

B. Descargar un proyecto: git clone

Clonar = descargar un repo completo (con todo su historial)

Es el equivalente a:

git init + **git remote add** + **git pull** (todo en un comando)

Se va a descargar una carpeta con un proyecto, tengan en cuenta descargarla en un lugar apropiado (si vienen siguiendo los pasos anteriores, salgan de la carpeta del proyecto anterior).

```
$ git clone git@github.com:alumnox/informatica-ii.git
Cloning into 'informatica-ii'...
remote: Enumerating objects: 42, done.
remote: Total 42 (delta 0), reused 42 (delta 0)
```

¿Cuándo usar clone?

- Para descargar un repo que ya existe en GitHub
- Cuando te sumás a un proyecto existente
- Solo necesitás tener permisos de lectura (repos públicos: cualquiera)

⚠ Si no tenés permisos de **escritura**, podés clonar pero no pushear

Git + GitHub: el flujo completo

20

Tu trabajo local se sincroniza con GitHub vía push y pull



Workflow diario

21

Configuración inicial (UNA VEZ POR REPO):

Puede ser:

- A. Subir un proyecto local a GitHub**
- B. Descargar un proyecto: git clone**

Uso diario:

```
$ git add .  
$ git commit -m "mensaje descriptivo"  
$ git push
```

Git recuerda: **main local** → **origin/main remoto**

Es como decirle una vez dónde queda tu casa, y después solo decís "llevame a casa".

git fetch vs git pull

¿Cómo traemos cambios que están en GitHub a nuestra PC?

git fetch

- Descarga los objetos y referencias del remoto (commits, ramas, tags)
- NO modifica tus archivos locales
- Es seguro, solo "mira"

git pull

- Hace fetch + merge automáticamente
- Descarga Y aplica los cambios
- Modifica tus archivos locales

Visualizar ramas remotas

Después de hacer fetch, podemos ver los commits remotos:

```
$ git log --oneline --decorate --all --graph
* a1b2c3d (origin/main) Agregar README
* d4e5f6g (HEAD -> main) Tercer commit
* h7i8j9k Segundo commit
* l0m1n2o Primer commit
```

origin/main — donde está GitHub (tiene un commit que no tenemos)

HEAD -> main — donde estamos nosotros (local)

Para traer los cambios: **git pull**

Fork: contribuir sin permisos de escritura

Fork = crear una copia del repo en TU cuenta de GitHub

- Es una función de GitHub (no de Git)
- Después clonás TU fork a tu PC
- Podés pushear a TU fork libremente



¿Qué es un Pull Request?

25

"Hola, hice estos cambios. ¿Los querés incorporar?"

Un Pull Request (PR) es una propuesta de cambios.

Es el mecanismo para pedir que tus cambios se integren en otro repositorio (o en otra rama).

Es una función de **GitHub**, no de Git

Revisión de código

Otros pueden ver y comentar tus cambios antes de aceptarlos

Discusión

Se puede discutir cada línea de código en el PR

Historial

Queda registro de quién cambió qué, cuándo y por qué

Fork + PR: el flujo completo

26



Este es el flujo estándar de contribución en proyectos open source

Fork vs Branch: ¿cuándo usar cada uno?

Branch + PR

(Trabajo en equipo)

- Tenés acceso de escritura al repo
- Creás una rama, hacés cambios
- PR contra main del mismo repo

Ejemplo:

Proyecto del equipo en la empresa

Fork + PR

(Contribución externa)

- NO tenés acceso de escritura
- Forkeás, clonás tu fork
- PR desde tu fork al repo original

Ejemplo:

Contribuir a un proyecto open source

Para el ejercicio de hoy: usamos Fork + PR

Buenas prácticas en Pull Requests

28

Título descriptivo

"Agregar usuario alumnox" en vez de "cambios"

Explicá el porqué, no el qué

El diff ya muestra qué cambiaste. Explicá por qué.

PRs chicos y enfocados

Un PR por tarea. No mezclar cosas sin relación.

Respondé a los comentarios

Si te piden cambios, hacelos y pusheá de nuevo.

Un buen PR facilita la revisión y acelera la integración.

En la materia, los TPs se evalúan vía PR. Cuanto más claro, mejor.

Ejercicio: contribuí al repo de la cátedra

29

1

1. Crear cuenta GitHub

(si no lo hicieron todavía)

2

2. Forkear el repo de Info II

Botón "Fork" en github.com/sfweber/informatica-ii

3

3. Clonar TU fork

```
git clone git@github.com:TU_USUARIO/informatica-ii.git
```

4

4. Editar PARTICIPANTS.md

Agregar tu usuario de GitHub al archivo

5

5. Subir los cambios

```
git add . → git commit -m "... " → git push
```

6

6. Crear Pull Request

Desde GitHub: botón "New Pull Request" en tu fork

Demo: creando un Pull Request

30

Hagamos el ejercicio juntos paso a paso:

```
# 1. Forkear el repo del profesor (desde GitHub)
$
# 2. Clonar TU fork
$ git clone git@github.com:ALUMNO/informatica-ii.git
$ cd informatica-ii
$
# 3. Editar el archivo PARTICIPANTS.md
$ nano PARTICIPANTS.md
# Agregar tu usuario de GitHub al archivo
$
# 4. Guardar y subir
$ git add PARTICIPANTS.md
$ git commit -m "Agregar mi usuario: ALUMNO"
$ git push
$
# 5. Crear Pull Request (desde GitHub)
```

Configuración (una vez por repo)

- A. Subir un proyecto local a GitHub
- B. Descargar un proyecto: `git clone`

Uso diario

```
git add .  
git commit -m "msg"  
git push  
git fetch  
git pull
```

Colaboración

Fork (GitHub UI)
Pull Request (GitHub UI)

Cheat Sheet: Git + GitHub

32



git remote -v — ver remotes configurados (muestra a qué URL está conectado tu repo local)

git log --oneline --all --graph — ver historial con ramas remotas y locales

Fork — copiar repo ajeno a tu cuenta (GitHub)

Pull Request — proponer cambios a otro repo (GitHub)

¿Preguntas?



A partir de ahora:

- Usen GitHub para TODOS sus proyectos
- **Los TPs se entregan vía Pull Request**
- Su perfil de GitHub es su portfolio profesional

docs.github.com/es